

Section 3: Emerging Technologies Skills Testing

1. Define entity recognition

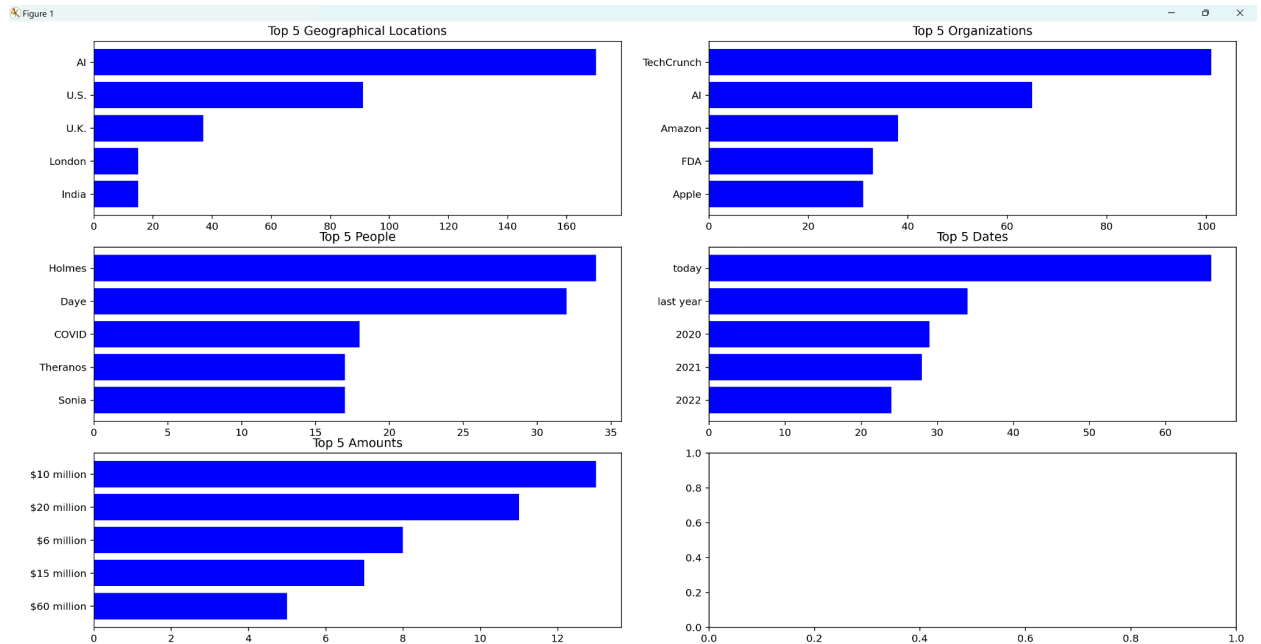
Entity recognition, also known as Named Entity Recognition (NER), is a sub-task of Natural Language Processing (NLP) that involves identifying and classifying key information (entities) in text into predefined categories such as names of people, organizations, locations, etc.

2. Based on your understanding of the concept of entity recognition, extract the following information from the text: geographical locations, organizations, people, dates, and amounts mentioned in the text. Present the results graphically.

Code: <https://github.com/waitheramwaurar/Entity-recognition/blob/main/extraction.py>

```
extraction.py > ...
1 import spacy
2 from collections import Counter
3 import matplotlib.pyplot as plt
4
5 # Loading the dataset
6 import pandas as pd
7 text_df = pd.read_csv("text_data - text_data.csv")
8
9 # Loading spaCy NLP library's English model
10 nlp = spacy.load("en_core_web_sm")
11
12 # Combine all stories into one text
13 all_text = " ".join(text_df["story"].dropna())
14
15 # Apply the NLP model
16 doc = nlp(all_text)
17
18 # Define entity labels of interest
19 labels_of_interest = {
20     "GPE": "Geographical Locations",
21     "ORG": "Organizations",
22     "PERSON": "People",
23     "DATE": "Dates",
24     "MONEY": "Amounts"
25 }
26
27 # Extract entities by category
28 entities = {label: [] for label in labels_of_interest.values()}
29 for entity in doc.ents:
30     if entity.label_ in labels_of_interest:
31         entities[labels_of_interest[entity.label_]].append(entity.text)
32
33 # Count and visualize top 5 entities in each category
34 fig, axes = plt.subplots(3, 2, figsize=(15, 12))
35 axes = axes.flatten()
36
37 for i, (category, values) in enumerate(entities.items()):
38     top_entities = Counter(values).most_common(5)
39     labels, counts = zip(*top_entities) if top_entities else ([], [])
40     axes[i].barh(labels, counts, color="blue")
41     axes[i].set_title(f"Top 5 {category}")
42     axes[i].invert_yaxis()
43
44 # Hide the unused subplot if odd number
45
46 plt.tight_layout()
47 plt.show()
```

Output:



3. Perform topic modelling on the text to identify key thematic areas that the authors discuss in the article.

Code:

<https://github.com/waitheramwaurar/Entity-recognition/blob/main/topicmodelling.py>

```

1  import pandas as pd
2  import spacy
3  from sklearn.feature_extraction.text import CountVectorizer
4  from sklearn.decomposition import LatentDirichletAllocation
5
6  # Loading the data
7  df = pd.read_csv("text_data - text_data.csv")
8  text_data = df['story'].dropna().astype(str)
9
10 # Preprocess the text using spaCy
11 nlp = spacy.load("en_core_web_sm")
12
13 def preprocess(doc):
14     doc = nlp(doc)
15     return " ".join([token.lemma_.lower() for token in doc if token.is_alpha and not token.is_stop])
16
17 cleaned_text = text_data.apply(preprocess)
18
19 # Vectorize the text using CountVectorizer
20 vectorizer = CountVectorizer(max_df=0.95, min_df=2, stop_words='english')
21 dtm = vectorizer.fit_transform(cleaned_text)
22
23 # Fit the LDA model
24 lda_model = LatentDirichletAllocation(n_components=5, random_state=42)
25 lda_model.fit(dtm)
26
27 # Display top words in each topic
28 words = vectorizer.get_feature_names_out()
29 topics = {}
30 for i, topic in enumerate(lda_model.components_):
31     topic_words = [words[i] for i in topic.argsort()[::-10:]]
32     topics[f"Topic #{i + 1}"] = topic_words
33
34 # Display the topics
35 print(topics)

```

Output:

```
{'Topic #1': ['year', 'technology', 'million', 'ai', 'work', 'new', 'base', 'protein', 'say', 'company'], 'Topic #2': ['patient', 'say', 'health'], 'Topic #3': ['new', 'raise', 'market', 'healthcare', 'year', 'health', 'million', 'device', 'sonia', 'say', 'treatment', 'woman', 'period'], 'Topic #4': ['sonia', 'say', 'treatment', 'woman', 'period', 'sonia', 'say', 'treatment', 'woman', 'period', 'pain'], 'Topic #5': ['datum', 'say', 'new', 'drug', 'work']}
```

4. Using transformer models, summarize the text into one precise paragraph.

Code: <https://github.com/waitheramwaurar/Entity-recognition/blob/main/summary.py>

```
summary.py > ...
1 import pandas as pd
2 from transformers import pipeline
3
4 # Load the dataset
5 df = pd.read_csv("text_data - text_data.csv")
6
7 from transformers import pipeline
8
9 # Load the pre-trained summarization model from HuggingFace
10 summarizer = pipeline("summarization")
11
12 # Define a function to summarize long texts by splitting into smaller chunks
13 def summarize_text(text, max_length=1024, summary_max_length=150):
14     # Split the text into chunks that fit the model's maximum token length
15     words = text.split()
16     chunk_size = max_length // 2 # Approximate size of each chunk
17     chunks = [words[i:i + chunk_size] for i in range(0, len(words), chunk_size)]
18
19     # Summarize each chunk
20     summaries = []
21     for chunk in chunks:
22         chunk_text = " ".join(chunk)
23         summary = summarizer(chunk_text, max_length=summary_max_length, min_length=50, do_sample=False)
24         summaries.append(summary[0]['summary_text'])
25
26     # Combine all summaries
27     final_summary = " ".join(summaries)
28     return final_summary
29
30 # Use the first article from the dataset as a sample
31 sample_story = df['story'].iloc[0]
32
33 # Summarize the text
34 summary = summarize_text(sample_story)
35
36 # Print the summary
37 print(summary)
```

Output:

```
Kaiser Permanente has signed a deal with French AI startup Nabla to roll out an AI assistant to doctors and other clinicians in its network . The aim is to reduce the amount of time doctors and others spend on admin . The service will be available to all physicians and clinicians, but using it will be optional . Doctors in the pilot cut out 1.5 hours of admin time using Nabla's Copilot, the company said . Nabla has raised just under $23 million from investors that include Tony Fadell, Firstminute Capital and Artemis .
```

5. Apply unsupervised machine learning techniques to classify the texts into the following categories, generating ratios for each class.

Categories: Advanced Gene Editing, mRNA Technology, CAR-T Cell Therapy, Organoids and Tissue Engineering, Single-Cell Genomics, Synthetic Biology, Biological Computing, Wearable Biosensors, Microbiome Therapeutics, Nanomedicine

Code: <https://github.com/waitheramwaurar/Entity-recognition/blob/main/categories.py>

```

categories.py > ...
1 import pandas as pd
2 import re
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from sklearn.cluster import KMeans
5 from sklearn.preprocessing import LabelEncoder
6
7 # Load the dataset
8 df = pd.read_csv("text_data - text_data.csv")
9
10 # Clean the text data
11 def basic_preprocess(doc):
12     # Remove non-alphabetic characters and convert to lowercase
13     doc = re.sub(r"[^a-zA-Z\s]", '', doc)
14     doc = doc.lower()
15     return doc
16
17 # Apply preprocessing
18 df['cleaned_text'] = df['story'].apply(basic_preprocess)
19
20 # Vectorize the text using Tf-IDF
21 vectorizer = TfidfVectorizer(stop_words='english', max_df=0.95, min_df=2)
22 X = vectorizer.fit_transform(df['cleaned_text'])
23
24 # Apply KMeans clustering
25 num_categories = 10 # Number of predefined categories
26 kmeans = KMeans(n_clusters=num_categories, random_state=42)
27 df['cluster'] = kmeans.fit_predict(X)
28
29 # Define the category labels (your predefined categories)
30 categories = ['Advanced Gene Editing', 'mRNA Technology', 'CAR-T Cell Therapy', 'Organoids and Tissue Engineering',
31              'Single-Cell Genomics', 'Synthetic Biology', 'Biological Computing', 'Wearable Biosensors',
32              'Microbiome Therapeutics', 'Nanomedicine']
33
34 # Encode the clusters with category labels
35 label_encoder = LabelEncoder()
36 df['category'] = label_encoder.fit_transform(df['cluster'])
37
38 # Map the cluster labels to the category names
39 df['category_name'] = df['category'].apply(lambda x: categories[x])
40
41 # Calculate the ratio of each category
42 category_counts = df['category_name'].value_counts(normalize=True) # Normalized to get the ratio
43 category_ratios = category_counts.to_dict()
44
45 # Display the results
46 print(category_ratios)
47 print(df[['story', 'category_name']].head()) # Display the first few stories with their assigned categories

```

Output:

```

category_name
mRNA Technology      0.186667
Biological Computing  0.166667
Nanomedicine         0.133333
CAR-T Cell Therapy   0.100000
Synthetic Biology     0.093333
Single-Cell Genomics  0.086667
Microbiome Therapeutics 0.073333
Advanced Gene Editing 0.066667
Wearable Biosensors  0.060000
Organoids and Tissue Engineering 0.033333

```

6. Extract and explain emerging trends within the industry

Emerging trends in biotechnology

mRNA Technology

mRNA technology is a method that uses messenger RNA (mRNA) to instruct cells to produce proteins, which can help treat diseases. This technology was used to create the

COVID-19 vaccines, and it's now being explored for other uses like treating cancer and other viral infections.

It represents a faster, more flexible way to create vaccines and potentially even treatments for various diseases. Its success in the pandemic has sparked a wave of research into its broader applications.

Advanced gene editing (e.g., CRISPR)

Advanced gene editing, like CRISPR, allows scientists to "edit" the DNA of living organisms. This can potentially fix genetic diseases, enhance crops, and even create personalized medicine.

It could lead to cures for genetic diseases that were once considered untreatable, like sickle cell anemia or cystic fibrosis. It also has applications in improving agricultural crops and livestock.

CAR-T cell therapy

CAR-T cell therapy is a treatment that modifies a patient's own immune cells (T cells) to better attack cancer cells. It's mainly used for certain types of blood cancers like leukemia and lymphoma.

It's one of the most promising cancer treatments available, as it targets cancer cells with a patient's own immune system. It's a breakthrough in personalized cancer therapy.

Organoids and tissue engineering

Organoids are small, simplified versions of organs grown in a lab, used to study diseases and test drugs. Tissue engineering involves growing human tissues and organs for medical use, potentially even transplanting them into patients.

These technologies could make it possible to test drugs more effectively without animal testing, and they could lead to the creation of lab-grown organs for transplant, addressing organ shortages.

Wearable biosensors

Wearable biosensors are devices that you wear on your body, like fitness trackers or health monitors, that continuously collect data about your body's health (e.g., heart rate, glucose levels).

These devices help people monitor their health in real-time, catch problems early, and manage chronic conditions like diabetes or heart disease. They're part of the move towards more personalized and preventative healthcare.

7. Draw conclusions.

The biotechnology field is experiencing unprecedented growth, with innovations that have the potential to radically improve healthcare and the quality of life. **mRNA technology, gene editing, CAR-T cell therapies, organ regeneration, and wearable health monitoring** systems are leading this transformation, offering new hope for treating diseases and improving human health. However, these advances must be carefully managed to address ethical, regulatory, and accessibility challenges.

The coming decades could see biotech lead not only in curing diseases but also in preventive care, personalized medicine, and the development of sustainable, efficient healthcare systems globally.